AI Assisted Coding

Assignment 7.5

Name: K.Chaitanya

Ht.no: 2303A51677

Batch: 22

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Lab Objectives:

• To identify and correct syntax, logic, and runtime errors in Python programs using AI tools.

• To understand common programming bugs and AI-assisted debugging suggestions.

• To evaluate how AI explains, detects, and fixes different types of coding errors.

• To build confidence in using AI to perform structured debugging practices.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

• Use AI tools to detect and correct syntax, logic, and runtime errors.

• Interpret AI-suggested bug fixes and explanations.

• Apply systematic debugging strategies supported by AI-

generated insights.

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument cause

unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

def add_item(item, items=[]):

items.append(item)

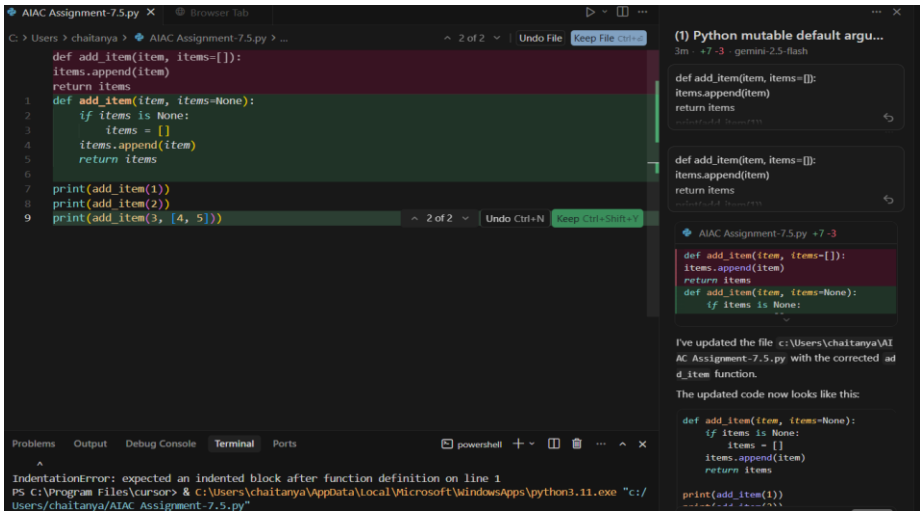
return items

print(add_item(1))

print(add_item(2))

Expected Output: Corrected function avoids shared list bug.

Code:



Output:

To avoid this, a common practice is to use None as the default value and then create a new list inside the function
if items is No

Observation:

This function correctly handles mutable default arguments.It uses None as the default for items and initializes a new list only if items is None.This prevents unintended shared state and accumulation of items across function calls.

Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails.

Use AI to correct with tolerance.

Bug: Floating point precision issue

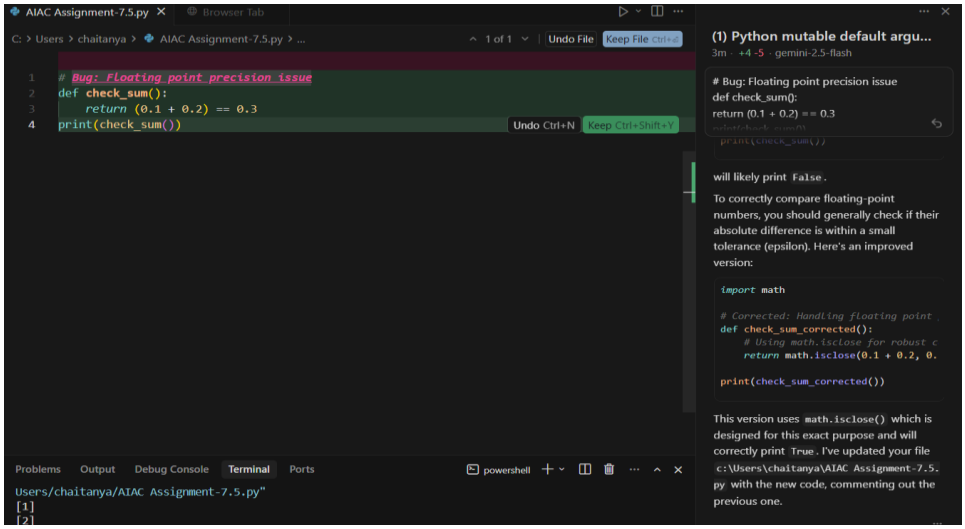
def check_sum():

return (0.1 + 0.2) == 0.3

print(check_sum())

Expected Output: Corrected function

Code:



Output:

```
PS C:\Program Files\cursor> & C:\Users\chaitanya\AppData\Local\Microsoft\WindowsApps\python3.11.exe
Users/chaitanya/AIAC Assignment-7.5.py"
False
PS C:\Program Files\cursor>
```

To correctly compare floating-point numbers, you should generally check if their absolute difference is within a small tolerance (epsilon). Here's an improved version:

Observation:

Observing correctly that (0.1 + 0.2) == 0.3 will likely evaluate to False due to the inherent limitations of floating-point representation in computers. Binary representations of decimal numbers like 0.1 and 0.2 are often inexact.

The sum 0.1 + 0.2 results in a value that is extremely close to, but not exactly 0.3, causing the direct equality check to fail. The standard solution is to compare floating-point numbers within a small tolerance using functions like math.isclose().

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

def countdown(n):

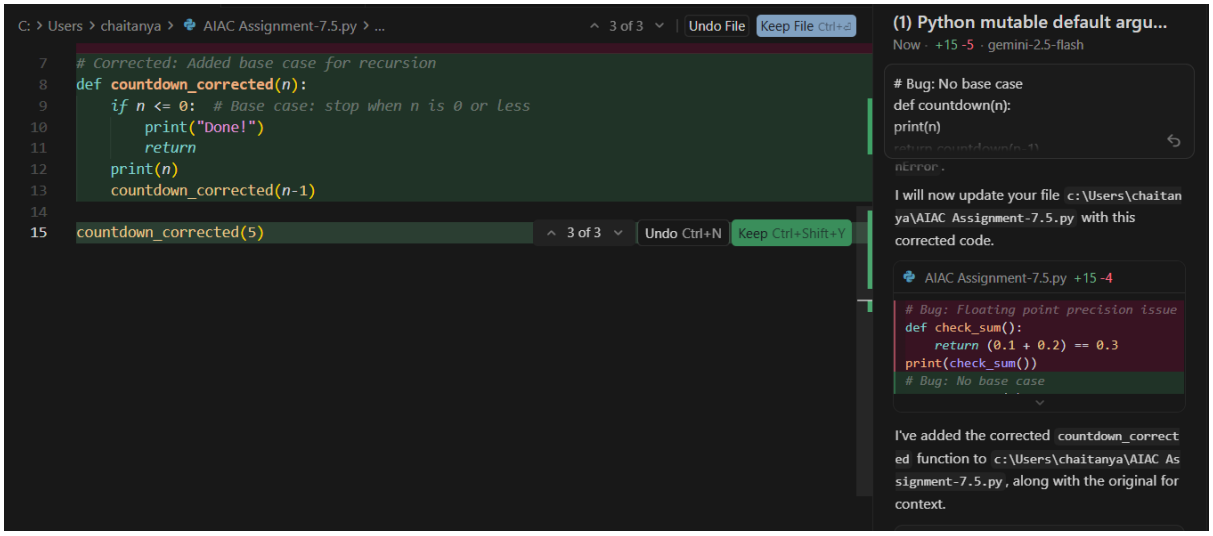
print(n)

return countdown(n-1)

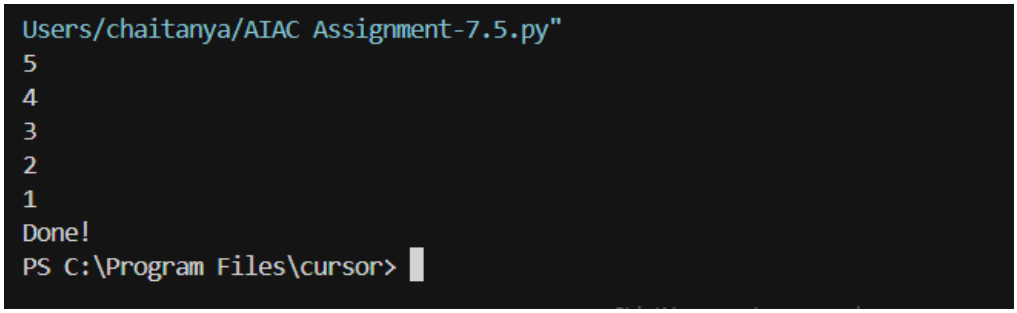
countdown(5)

Expected Output : Correct recursion with stopping condition.

Code:



Output:



Observation:

The recursion error occurred due to the absence of a base case, which caused infinite function calls.
Using AI, a proper stopping condition was added to terminate recursion when the value reached zero.
This fix ensured correct execution and highlighted the importance of base cases in recursive functions.

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes

error. Use AI to fix it.

Bug: Accessing non-existing key

def get_value():

data = {"a": 1, "b": 2}

return data["c"]

print(get_value())

Expected Output: Corrected with .get() or error handling.

Code:

```python
# Task 4 (Dictionary Key Error)
'''
Task: Analyze given code where a missing dictionary key causes
error. Use AI to fix it.
# Bug: Accessing non-existing key
Expected Output: Corrected with .get() or error handling.
'''
def get_value():
    data = {"a": 1, "b": 2}
    return data["c"]
    return data.get("c", "Key not found")
print(get_value())
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Key not found
```

Observation:

The error occurred because the program tried to access a dictionary key that does not exist, which raised a KeyError. Using AI, the issue was fixed by replacing direct key access with the safer .get() method or by applying proper error handling.
This task highlighted the importance of validating keys to prevent runtime crashes and improve program reliability.

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect

and fix it.

Bug: Infinite loop

def loop_example():

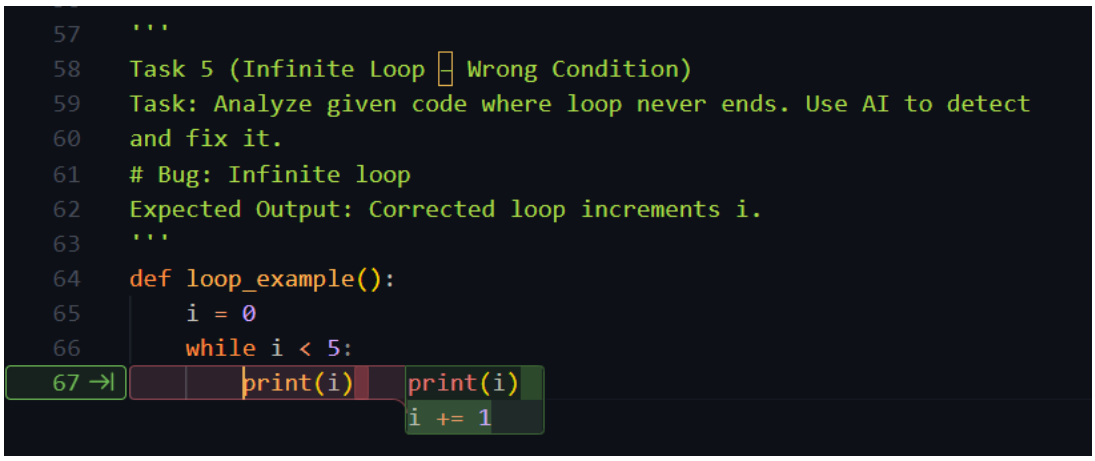
i = 0

while i < 5:

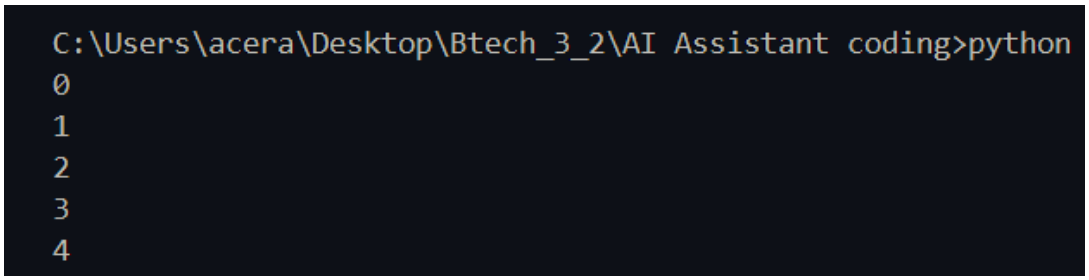
print(i)

Expected Output: Corrected loop increments i.

Code:



Output:



Observation:

The infinite loop occurred because the loop control variable i was never incremented inside the while loop.
Using AI, the issue was identified and fixed by adding i += 1 to ensure the loop progresses toward termination.
This task emphasized the importance of properly updating loop variables to avoid non-terminating programs.

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to

fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

Code:

```
71  # Task 6 (Unpacking Error – Wrong Variables)
72  '''
73  Task: Analyze given code where tuple unpacking fails. Use AI to
74  fix it.
75  # Bug: Wrong unpacking
76  Expected Output: Correct unpacking or using _ for extra values.
77  '''
78  a, b = (1, 2, 3)
    a, b, _ = (1, 2, 3)
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
1 2
```

Observation:

The unpacking error happened because the number of variables didn’t match the number of elements in the tuple.
AI suggested either matching the variables to the elements or using a placeholder like _ for extra values.
This task highlighted the need for careful alignment in unpacking

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

def func():

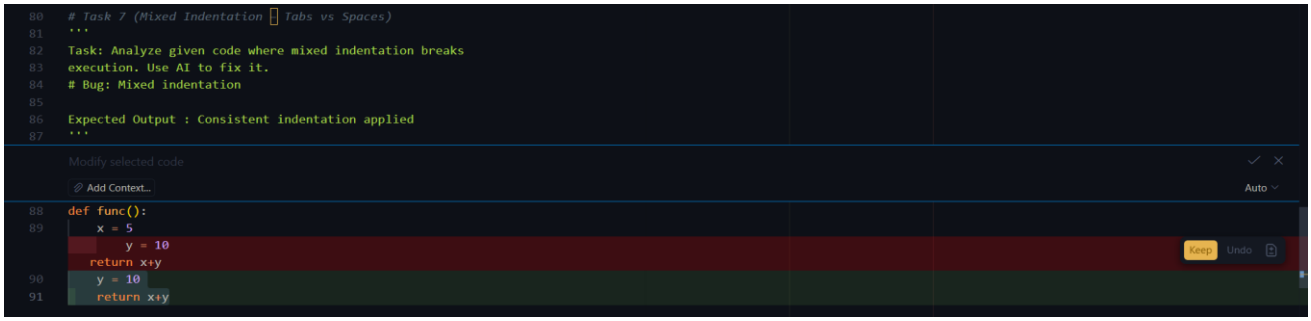
x = 5

y = 10

return x+y

Expected Output : Consistent indentation applied.

Code:



Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
15
```

Observation:

The error occurred due to inconsistent indentation, where tabs and spaces were mixed, causing an IndentationError.
Using AI, the code was corrected by applying consistent indentation (preferably four spaces).
This task highlighted the importance of maintaining uniform indentation for proper Python execution and readability.

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

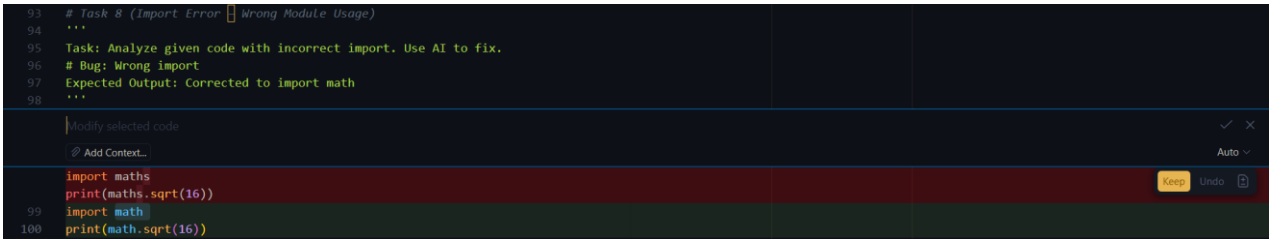
Bug: Wrong import

import maths

print(maths.sqrt(16))

Expected Output: Corrected to import math

Code:



Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
4.0
```

Observation:

The error occurred because the program attempted to import a non-existent module maths, resulting in a ModuleNotFoundError.
Using AI, the import was corrected to the proper built-in module math, allowing math.sqrt(16) to execute successfully.
This task emphasized the importance of using correct module names and understanding standard library usage in Python.

Observation:

- This lab helped us identify and fix different types of Python errors such as logical, runtime, syntax, recursion, and import errors using AI tools.
- AI explained the root causes clearly, including issues like mutable default arguments, floating-point precision problems, and missing base cases in recursion.
- We learned best practices such as using tolerance for float comparison, safe dictionary access with .get(), proper loop control, and correct tuple unpacking.
- Overall, the lab improved our systematic debugging approach and increased confidence in using AI for structured and efficient error correction.